

Cafe inventory management Design specifications

Version: 1.3

Created date: 2026/08/18

Author: Phusadeekunpaisan Nuttachai

1. System overview

1.1. System name

Cafe inventory management

1.2. Purpose

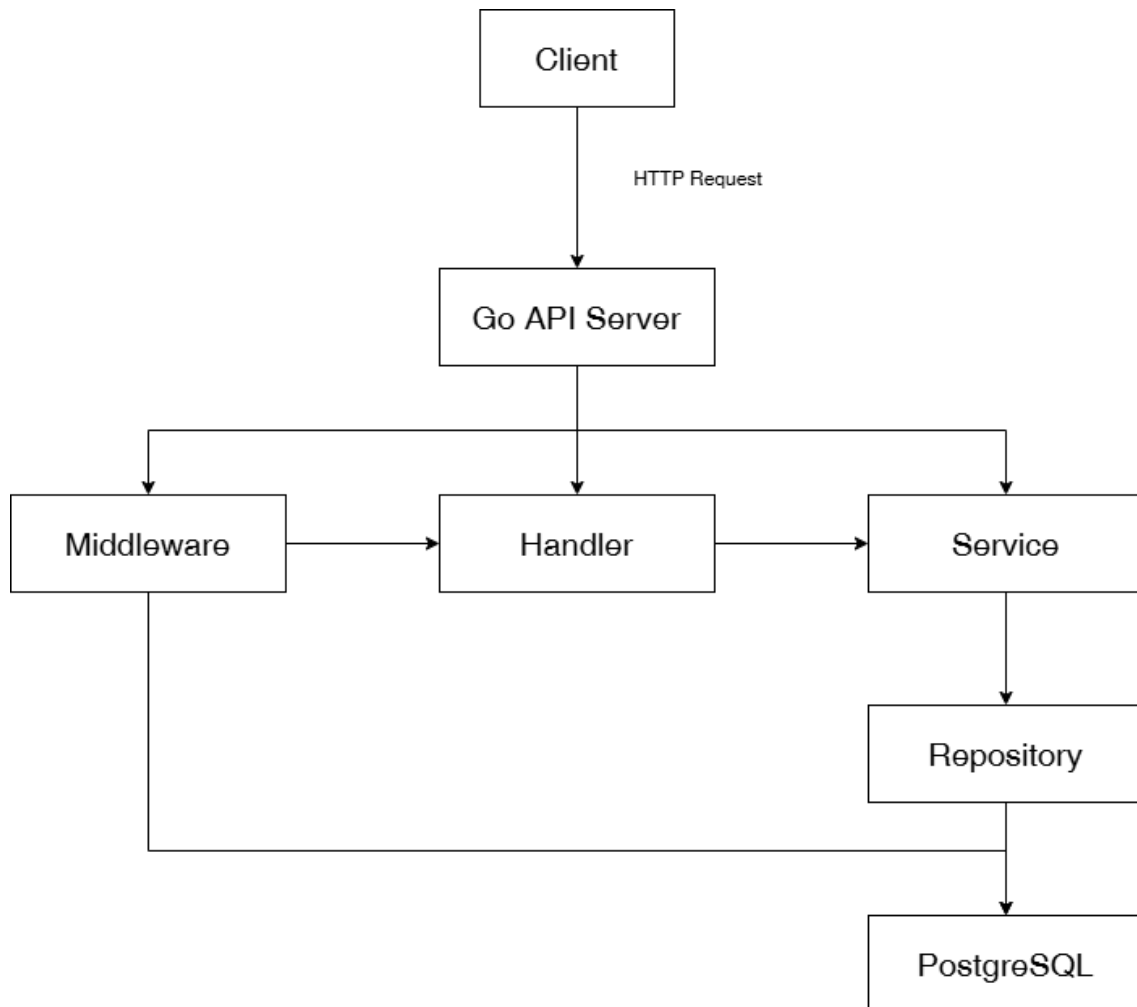
A production-style RESTful backend API built with Go(`net/http`) and PostgreSQL for managing a multi-branch cafe chain.

The system is designed for companies that operate multiple cafe branches under a single brand. Headquarters can centrally manage cafes, products, users, and inventory, while branch staff update stock levels in real time. The project also provides location-based search features to help customers find nearby cafes and available products.

1.3. Development enviornment

item	content
Language	Go
HTTP server	net/http
Database	PostgreSQL
Database Driver	pgx
Authentication	JWT
Password Hashing	bcrypt
Container	Docker
Architecture	Layered Architecture
API Style	RESTful API
Infrastructure	AWS ECS
Version Control	Git & GitHub

2. System structure



3.Function list

FN ID	FN name	details	Authorization
CAFE-01	Cafe registration	Register a new cafe Information	ADMIN
CAFE-02	Get cafe details	Get detailed information about the specified cafe.	All user
CAFE-03	Get a list of cafes	Get a list of cafes	All user
CAFE-04	Cafe update	Update the cafe information	ADMIN
CAFE-05	Cafe delete	Delete the cafe information	ADMIN
CATE-01	Category registration	Register a new category Information	ADMIN
CATE-02	Get category details	Get detailed information about the specified category.	All user
CATE-03	Get a list of categories	Get a list of categories	All user
CATE-04	Category update	Update the category information	ADMIN

CATE-05	Category delete	Delete the category information	ADMIN
PROD-01	Product registration	Register a new product Information	ADMIN
PROD-02	Get product details	Get detailed information about the specified product.	All user
PROD-03	Get a list of products	Get a list of products	All user
PROD-04	Product update	Update the product information	ADMIN
PROD-05	Product delete	Delete the product information	ADMIN
INV-01	Get inventory detail	Get current information about specified inventory	All user
INV-02	Get inventory list	Get a list of inventories	All user
INV-03	Inventory update	Update specified inventory quantity	STAFF / ADMIN
INV-04	Inventory history retrieval	Retrieve inventory change history	STAFF / ADMIN
USER-01	User registration	Register a new user Information	ADMIN
USER-02	Get user details	Get detailed information about	ADMIN

		the specified user.	
USER-03	Get a list of users	Get a list of users	ADMIN
USER-04	User update	Update the user information	ADMIN
USER-05	User delete	Delete the user information	ADMIN
AUTH-01	Login	Authenticate using email address and password then issue a JWT.	All user

4.DB design

4.1. cafes

Item	Type	PK	FK	NULL	Detail
id	SERIAL	○		×	cafe id
name	VARCHAR(255)			×	cafe name
address	TEXT			×	address
latitude	DECIMAL(9,6)			×	latitude
longitude	DECIMAL(9,6)			×	longitude
nearest_station	VARCHAR(255)			○	nearest station
opening_time	TIME			○	opening time
closing_time	TIME			○	closing time
created_at	TIMESTAMP			×	created time

updated_at	TIMESTAMP			×	last update time
-------------------	-----------	--	--	---	---------------------

4.2. products

Item	Type	PK	FK	NULL	Detail
id	SERIAL	○		×	product id
cafe_id	INT		○	×	cafe id
category_id	INT		○	×	category id
name	TEXT			×	product name
description	TEXT			○	product detail
price	NUMERIC(10,2)			○	product price
is_active	BOOLEAN			×	enable flag
created_at	TIMESTAMP			×	created time
updated_at	TIMESTAMP			×	last updated time

4.3. categories

Item	Type	PK	FK	NULL	Detail
id	SERIAL	○		×	category id
name	VARCHAR(100)			×	category name
created_at	TIMESTAMP			×	created time
updated_at	TIMESTAMP			×	last updated time

Note

Due to the UNIQUE constraint, name must not be duplicated.

4.4. inventory

Item	Type	PK	FK	NULL	Detail
id	SERIAL	○		×	inventory id
product_id	INT		○	×	product id
stock_quantity	INT			×	stock quantity
created_at	TIMESTAMP			×	created time
updated_at	TIMESTAMP			×	last updated time

Note

- Due to the UNIQUE constraint, product_id must not be duplicated.
- The inventory count is recorded in inventory_logs each time it is updated.
- stock_quantity must not be less than 0.

4.5. users

Item	Type	PK	FK	NULL	Detail
id	SERIAL	○		×	user id
username	VARCHAR(100)			×	username
email	VARCHAR(255)			×	email
password_hash	TEXT			×	password hash
user_role	VARCHAR(10)			×	user role
is_active	BOOLEAN			×	enable flag
created_at	TIMESTAMP			×	created time
updated_at	TIMESTAMP			×	last updated time

Note

Due to the UNIQUE constraint, email addresses must not be duplicated.

user_role (constraint)

item	detail
ADMIN	管理者
STAFF	スタッフ

4.6. inventory_logs

Item	Type	PK	FK	NULL	Detail
id	SERIAL	○		×	log id
inventory_id	INT		○	×	inventory id
user_id	INT		○	×	user id
change_quantity	INT			×	Changed Quantity
operation	VARCHAR(6)			×	operation type
created_at	TIMESTAMP			×	created time

operation (constraint)

item	detail
IN	Warehousing
OUT	Withdraw supplies
ADJUST	Stock adjustment

5.API design

5.1. Cafe registration

Endpoint (Authentication: Required)

```
POST /api/v1/cafes
```

Request

```
{  
  "name": "Tokyo Cafe",  
  "address": "Shinjuku",  
  "latitude": 35.6580,  
  "longitude": 139.7016,  
  "nearest_station": "Shinjuku station",  
  "opening_time": "08:00",  
  "closing_time": "21:00"  
}
```


Response

```
{  
  "id": 1,  
  "message": "Cafe created successfully"  
}
```

5.2. Get cafe details

Endpoint (Authentication: Not required)

```
GET /api/v1/cafes/{id}
```

Response

```
{  
  
  "id": 1,  
  
  "name": "Tokyo Cafe",  
  
  "address": "Shinjuku",  
  
  "latitude": 35.6580,  
  
  "longitude": 139.7016,  
  
  "nearest_station": " Shinjuku station",  
  
  "opening_time": "08:00",  
  
  "closing_time": "21:00"  
  
  "created_at": "2006-01-02 15:04:05",  
  
  "updated_at": "2006-01-02 15:04:05"  
  
}
```

5.3. Get a list of cafes

Endpoint (Authentication: Not required)

GET /api/v1/cafes

Query Parameters

item	type	detail
station	string	nearest station
limit	int	Maximum number of records to display
lat	float64	Latitude as a search criterion
lng	float64	Longitude as a search criterion
radius	float64	Search radius from lat/lng

Response

```
[
  {
    "id": 1,
    "name": "Tokyo Cafe",
    "address": "Shinjuku",
    "latitude": 35.6580,
    "longitude": 139.7016,
    "nearest_station": "Shinjuku station",
    "opening_time": "08:00",
    "closing_time": "21:00",
    "created_at": "2026-01-02 15:04:05",
    "updated_at": "2026-01-02 15:04:05"
  }
]
```

5.4. Cafe update

Endpoint (Authentication: Required)

```
PATCH /api/v1/cafes/{id}
```

5.5. Cafe delete

Endpoint (Authentication: Required)

```
DELETE /api/v1/cafes/{id}
```

Error response

409 Conflict

```
{
```

```
  "error": "cannot delete: other records still depend on this
```

```
item"
```

```
}
```

5.6. Category registration

Endpoint (Authentication: Required)

```
POST /api/v1/categories
```

Request

```
{  
  
  "name": "Drink",  
  
}
```

Response

```
{  
  
  "id": 1,  
  
  "message": "Category created successfully"  
  
}
```

Error response

409 Conflict

{

"error": "this category name is already used: <name>"

}

5.7. Get category details

Endpoint (Authentication: Not required)

```
GET /api/v1/categories/{id}
```

Response

```
{  
  "id": 1,  
  "name": "Drink",  
  "created_at": "2006-01-02 15:04:05",  
  "updated_at": "2006-01-02 15:04:05"  
}
```


5.8. Get a list of categories

Endpoint (Authentication: Not required)

GET /api/v1/categories

Query Parameters

item	type	detail
limit	int	Maximum number of records to display

Response

```
[  
  {  
    "id": 1,  
    "name": "Drink",  
    "created_at": "2026-01-02 15:04:05",  
    "updated_at": "2026-01-02 15:04:05"  
  }  
]
```

5.9. Category update

Endpoint (Authentication: Required)

```
PATCH /api/v1/categories/{id}
```

Error response

409 Conflict

```
{
```

```
  "error": "this category name is already used: <name>"
```

```
}
```

5.10. Category delete

Endpoint (Authentication: Required)

```
DELETE /api/v1/categories/{id}
```

Error response

409 Conflict

{

 "error": "cannot delete: other records still depend on this

item"

}

5.11. Product registration

Endpoint (Authentication: Required)

```
POST /api/v1/products
```

Request

```
{  
  "cafe_id": 1,  
  "category_id": 1,  
  "name": "Latte",  
  "description": "Hot latte",  
  "price": 480.00  
}
```

Response

```
{  
  "id": 1,  
  "message": "Product created successfully"  
}
```

5.12. Get product details

Endpoint (Authentication: Not required)

```
GET /api/v1/products/{id}
```

Response

```
{  
  "id": 1,  
  "cafe_id":1,  
  "cafe_name": "Tokyo Cafe",  
  "category_id":1,  
  "category_name": "Drink",  
  "name": "Latte",  
  "description": "Hot Latte",  
  "price": 480.00,  
  "is_active": true,  
  "created_at": "2026-01-02 15:04:05",  
  "updated_at": "2026-01-02 15:04:05"  
}
```


5.13. Get a list of products

Endpoint (Authentication: Not required)

```
GET /api/v1/cafes/{cafeId}/products
```

Response

```
[  
  {  
    "id": 1,  
    "cafe_id":1,  
    "cafe_name": "Tokyo Cafe",  
    "category_id":1,  
    "category_name": "Drink",  
    "name": "Latte",  
    "description": "Hot Latte",  
    "price": 480.00,  
    "is_active": true,  
    "created_at": "2006-01-02 15:04:05",  
    "updated_at": "2006-01-02 15:04:05"  
  }  
]
```

Query Parameters

item	type	detail
name	string	product name
limit	int	limit of record to be displayed

5.14. Product update

Endpoint (Authentication: Required)

```
PATCH /api/v1/products/{id}
```

5.15. Product delete

Endpoint (Authentication: Required)

```
DELETE /api/v1/products/{id}
```

5.16. Get inventory detail

Endpoint (Authentication: Not required)

```
GET /api/v1/inventory/{productId}
```

Response

```
{  
  
  "product_name": "Matcha cake",  
  
  "cafe_id": 1,  
  
  "cafe_name": "Tokyo cafe",  
  
  "stock_quantity": 50,  
  
  "updated_at": "2006-01-02 15:04:05"  
  
}
```

5.17. Get inventory list

Endpoint (Authentication: Not required)

GET /api/v1/inventory

Query Parameters

item	type	detail
name	string	product name
limit	int	Maximum number of records to display

Response

```
[  
  {  
    "id": 1,  
    "product_id": 1,  
    "product_name": "Matcha Cake",  
    "cafe_id": 1,  
    "cafe_name": "Tokyo cafe",  
    "stock_quantity": 40,  
    "updated_at": "2006-01-02 15:04:05"  
  },  
]
```

5.18. Inventory update

Endpoint (Authentication: Required)

```
PATCH /api/v1/inventory/{productId}
```

Request

```
{  
  "change_quantity": 5,  
  "operation": "OUT"  
}
```

Processing Flow

1. Update inventory.stock_quantity.
2. Register the operation history in inventory_logs.
3. Return the latest inventory quantity after the update.

Response

```
{  
  
  "message": "Inventory updated",  
  
  "stock_quantity": 45  
}
```

Request (Adjust)

```
{  
  
  "change_quantity": 5,  
  
  "operation": "ADJUST"  
}
```

Response

```
{  
  
  "message": "Inventory updated",  
  
  "stock_quantity": 5  
}
```


Note

In the case of ADJUST, change_quantity represents the absolute post-update inventory quantity itself, not a difference (delta). In the example above, it is set to 5 regardless of the pre-update inventory quantity.

Error response

409 Conflict

{

"error": "the stock doesnt have enough items"

}

5.19. Inventory history retrieval

Endpoint (Authentication: Required)

GET /api/v1/inventory/logs

Query Parameters

item	type	detail
inventory_id	int	inventory ID
user_id	int	user ID
operation	string	operation type
from	date	start date
to	date	end date
limit	int	Maximum number of records to display

Response

```
[  
  {  
    "id": 1,  
    "inventory_id": 1,  
    "user_id": 1,  
    "operation": "OUT",  
    "change_quantity": 5,  
    "created_at": "2006-01-02 15:04:05"  
  },  
]
```

5.20. User registration

Endpoint (Authentication: Required)

```
POST /api/v1/users
```

Request

```
{  
  "username": "staff01",  
  "email": "staff@example.com",  
  "password": "password01",  
  "user_role": "STAFF"  
}
```

Response

```
{  
  
  "id": 1,  
  
  "message": "User created successfully"  
  
}
```

Error response

```
409 Conflict  
  
{  
  
  "error": "this email is already used: <email>"  
  
}
```

5.21. Get user details

Endpoint (Authentication: Required)

```
GET /api/v1/users/{id}
```

Response

```
{  
  
  "id": 1,  
  
  "username": "staff01",  
  
  "email": "staff@example.com",  
  
  "user_role": "STAFF",  
  
  "is_active": true,  
  
  "created_at": "2026-01-02 15:04:05",  
  
  "updated_at": "2026-01-02 15:04:05"  
}
```

5.22. Get a list of users

Endpoint (Authentication: Required)

```
GET /api/v1/users
```

Response

```
[  
  {  
    "id": 1,  
    "username": "staff01",  
    "email": "staff@example.com",  
    "user_role": "STAFF",  
    "is_active": true,  
    "created_at": "2026-01-02 15:04:05",  
    "updated_at": "2026-01-02 15:04:05"  
  }  
]
```

Query Parameters

item	type	detail
limit	int	Maximum number of records to display

5.23. User update

Endpoint (Authentication: Required)

```
PATCH /api/v1/users/{id}
```

Error response

409 Conflict

```
{  
  
  "error": "this email is already used: <email>"  
}
```


5.24. User delete

Endpoint (Authentication: Required)

```
DELETE /api/v1/users/{id}
```

5.25. User authentication

Endpoint (Authentication: Not required)

```
POST /api/v1/auth/login
```

Request

```
{  
  
  "email": "admin@example.com",  
  
  "password": "password01"  
}
```

Response

```
{  
  
  "token": "jwt_token"  
}
```

Error response

401 Unauthorized

{

"error": "invalid email or password"

}

Error response

401 Unauthorized

{

"error": "this user account has been deactivated"

}

HTTP status code

status	detail
200 OK	completed successfully
201 Created	registration successful
204 No Content	deletion successful
400 Bad Request	request invalid
401 Unauthorized	authentication error
403 Forbidden	lack of authority
404 Not Found	no resource exists
409 Conflict	data duplication and conflict
500 Internal Server Error	internal server error

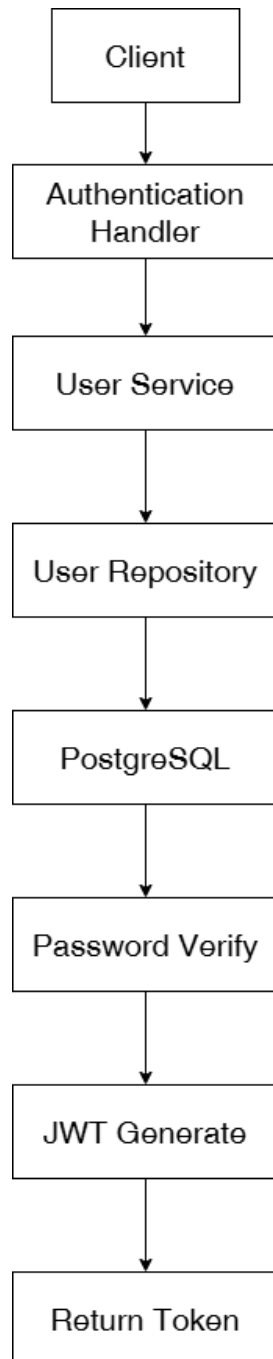
6.Authorization design

6.1. Authentication method

This system uses JWT (JSON Web Token) authentication.

The user logs in with an email address and password, configures authentication, and issues a JWT. For subsequent APIs that require authentication, set JWT in the HTTP request header to access.

6.2. Authentication flow



6.3. Authentication target API

API	Method	Authorization
/cafes	POST	ADMIN
/cafes/{id}	GET	Not required
/cafes	GET	Not required
/cafes/{id}	PATCH	ADMIN
/cafes/{id}	DELETE	ADMIN
/categories	POST	ADMIN
/categories/{id}	GET	Not required
/categories	GET	Not required
/categories/{id}	PATCH	ADMIN
/categories/{id}	DELETE	ADMIN
/products	POST	ADMIN
/products/{id}	GET	Not required
/products	GET	Not required
/products/{id}	PATCH	ADMIN

/products/{id}	DELETE	ADMIN
/inventory/{productId}	GET	Not required
/inventory	GET	Not required
/inventory/{productId}	PATCH	STAFF/ADMIN
/inventory/logs	GET	STAFF/ADMIN
/users	POST	ADMIN
/users/{id}	GET	ADMIN
/users	GET	ADMIN
/users/{id}	PATCH	ADMIN
/users/{id}	DELETE	ADMIN
/auth/login	POST	Not required

6.4. Authentication Failure Response

Authentication failed

HTTP Status

401 Unauthorized

Response

```
{  
  
  "error": "Invalid or expired token"  
  
}
```

Lack of authority

HTTP Status

403 Forbidden

Response

```
{  
  "error": "Permission denied"  
}
```

7.Directory structure

```
cafe-inventory-api/
├── .github/
│   └── workflows/
│       └── ci.yml
├── cmd/
│   └── server/
│       └── main.go
├── internal/
│   ├── handler/           // HTTP request
│   ├── service/          // Business logic
│   ├── repository/       // PostgreSQL access
│   ├── model/            // Domain model
│   ├── middleware/       // Common middleware
│   ├── router/           // Routing setting
│   ├── database/         // Connect to database
│   └── utils/            // Common utility
├── migrations/           // SQL migration
├── Dockerfile
├── .env
├── .gitignore
├── docker-compose.yml
├── go.mod
├── go.sum
└── README.md
```